Knapsack Problem

AlgoExpert · Hard · Dynamic Programming

The problem

- Items are [value, weight].
- Returns [[maxValue], [item indices]].

Intuition

- Each item is taken or skipped; a table over items and capacities captures every choice.

Approach

- 1. Row per item, column per capacity
- 2. Skip copies the row above
- 3. Take adds the value to the row above at capacity minus weight
- 4. Backtrack to list items

Java solution

```java
public class KnapsackProblem {
    public static List<List<Integer>> knapsackProblem(int[][] items, int capacity) {
        int n = items.length;
        int[][] dp = new int[n + 1][capacity + 1];
        for (int i = 1; i <= n; i++) for (int c = 0; c <= capacity; c++) {
            dp[i][c] = dp[i - 1][c];
            int w = items[i - 1][1];
            if (w <= c) dp[i][c] = Math.max(dp[i][c], dp[i - 1][c - w] + items[i - 1][0]);
        }
        List<Integer> chosen = new ArrayList<>();
        for (int i = n, c = capacity; i > 0; i--)
            if (dp[i][c] != dp[i - 1][c]) { chosen.add(0, i - 1); c -= items[i - 1][1]; }
        return List.of(List.of(dp[n][capacity]), chosen);
    }
}
```

Complexity

- Time: O(n*c)
- Space: O(n*c)

Recap

- Each item is taken or skipped; a table over items and capacities captures every choice.
- Complexity: O(n*c)